

Automotive Concepts

Android Automotive OS and the car around it

"If you can explain it with a real car in front of you, you understand it."

- 17 chapters, each tied to an animated lesson
- real cars, real standards, real APIs
- exercises with answers at the end

A Head First-style learning guide that goes with the **Automotive Academy** sample app (Android Automotive OS, Kotlin, Jetpack Compose). Generated September 2026.

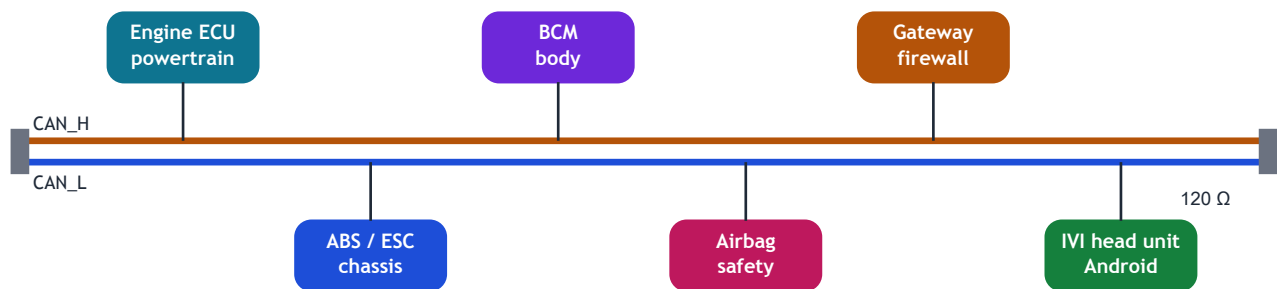
What's in this book

1 Your car is a computer network on wheels	3
E/E architecture: three generations	3
2 Android Automotive OS architecture	4
Android Auto vs Android Automotive OS	5
GAS or no GAS?	5
Boot in 30 seconds	5
3 The Vehicle HAL and vehicle properties	6
Using CarPropertyManager	7
4 HVAC and area properties	8
5 The instrument cluster and functional safety	9
ISO 26262 in one table	10
6 Driver distraction and UX restrictions	11
7 Media apps in the car	12
8 Car audio: contexts, zones and focus	13
9 Vehicle networks and diagnostics	14
Arbitration, bit by bit	14
Signals and DBC files	15
Diagnostics	15
10 Cameras, park assist, EVS and ADAS	16
11 Electric vehicles and charging	17
Why fast charging slows down after 80 %	17
12 Power management	18
13 Over-the-air (OTA) updates	19
14 Multiple displays, users and occupant zones	20
15 The Car App Library	21
16 Connectivity: Bluetooth, projection, telematics, V2X	22
17 AUTOSAR, SDV and how cars are tested	22
Sharpen your pencil: answers	23
Glossary	23

1 Your car is a computer network on wheels

70-150 computers, one very long ethernet cable... well, almost

A modern car contains **70 to 150 ECUs** (Electronic Control Units): small computers that each own one job: engine, brakes (ABS/ESC), airbags, doors and lights (Body Control Module), battery management, cameras, and the one you will work on: the **infotainment head unit (IVI)**.



ECUs share buses; a gateway separates domains

E/E architecture: three generations

Generation	Idea	Example
Distributed	One ECU per function; hundreds of wires; features fixed at the factory	Most cars before ~2015
Domain	Group ECUs into domains (powertrain, chassis, body, infotainment, ADAS) with a domain controller	VW MEB, many 2018-2023 platforms
Zonal / central compute	A few powerful central computers + zone controllers that handle wiring by location; Ethernet backbone; software defines features	Tesla, VW SSP (planned), GM Ultifi-style SDV

REAL LIFE

The **Software-Defined Vehicle (SDV)** is why companies like Harman, Bosch and KPIT hire so many Android and Linux engineers: features (new driving modes, apps, subscriptions like heated seats) are now delivered by software and OTA, not by adding hardware.

BRAIN POWER

If there are 100 ECUs and each is from a different supplier, who guarantees that the brake ECU and the infotainment ECU agree on what message `0x0Fo` means? (Answer: the OEM's communication matrix / DBC file, chapter 9.)

THERE ARE NO DUMB QUESTIONS

Q: Is the head unit a safety-critical ECU?

A: Usually no: IVI is 'QM' (quality managed, no ASIL). That is exactly why it runs a big OS like Android and why it must be **isolated** from safety ECUs by a gateway. The cluster, when it shows warning lamps, is safety-relevant.

Q: What is a Tier-1?

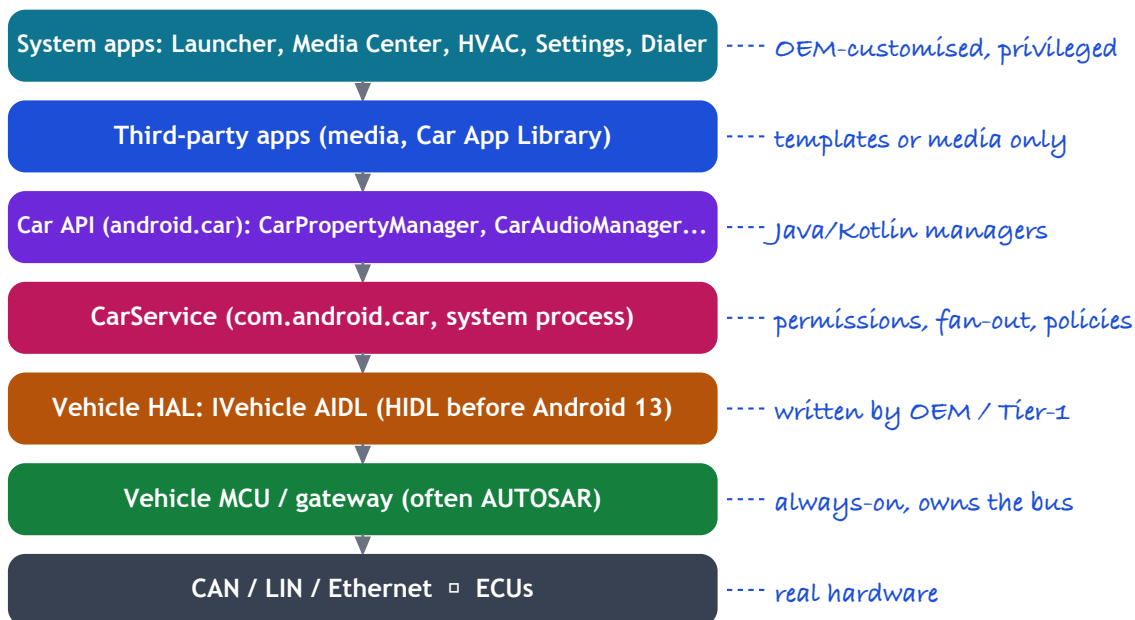
A: A company that supplies complete systems directly to car makers (OEMs): Harman, Bosch, Continental, Visteon, Aptiv, Denso. Tier-2 supplies parts/chips to Tier-1s (e.g. Qualcomm, NXP, Renesas).

BULLET POINTS

- ECU = embedded computer for one function.
- Domains → zonal → central compute.
- IVI is non-safety (QM); a gateway protects safety domains.
- OEM = car maker; Tier-1 = system supplier (Harman); Tier-2 = chips.

2 Android Automotive OS architecture

Android, plus a car-shaped layer cake



The AAOS stack. Everything above VHAL is Android; everything below is the car.

SEE IT MOVE: IN THE APP

Open lesson 1 **AAOS Architecture**. A packet travels down the layers (request: set driver temperature) and then back up (event: ECU confirms). Tap a layer to read what it does.

Android Auto vs Android Automotive OS

FIRESIDE CHAT

Tonight's talk: Android Auto vs Android Automotive OS

Android Auto: I live on your phone. When you plug in or connect wirelessly, I project a car-friendly UI onto the car screen. The car is just a monitor + touch + mic for me.

AAOS: I *am* the car's operating system. No phone needed. I boot with the car, I read speed and battery from VHAL, I control the climate.

Android Auto: But I update with the phone, so I'm always fresh!

AAOS: True, but I update by OTA and I can do things you never can: HVAC, seat settings, EV routing with real battery data, rear-seat displays.

Both: And developers can write one media app or one Car App Library app that runs on both of us.

	Android Auto	Android Automotive OS
Runs on	Phone (projection)	Head unit (embedded)
Needs a phone	Yes	No
Vehicle data (VHAL)	Very limited (via host)	Full, permission-controlled
App types	Media, messaging, Car App Library	Same + system apps built by OEM
Updated by	Play Store on the phone	OTA from the OEM + Play Store in car
Examples	Almost any car with a USB port	Polestar 2, Volvo EX30/XC40, GM, Renault, Honda Accord 2023

GAS or no GAS?

Google Automotive Services (GAS) = Google Maps, Assistant, Play Store licensed by the OEM ("Google built-in"). OEMs without GAS (e.g. some Chinese OEMs, or those choosing their own ecosystem) ship AAOS with their own maps, voice and app store.

Boot in 30 seconds



WATCH IT!

The rear camera and warning chimes cannot wait for Zygote. They are started early from native services (EVS, early audio) so they work within legal time limits.

AAOS runs a **headless system user** (user 0) in the background; the driver uses a secondary user (10, 11...). Code that assumes user 0 is the foreground user breaks on AAOS.

THERE ARE NO DUMB QUESTIONS

Q: Is CarService an app?

A: It is a privileged system app (package com.android.car / updatable car module) running in its own process, bound by system_server at boot. Apps talk to it through binder via the Car API managers.

Q: Why not let apps talk to VHAL directly?

A: Permissions, multiplexing (one VHAL, many clients), rate control, and survivability: CarService can restart without apps crashing.

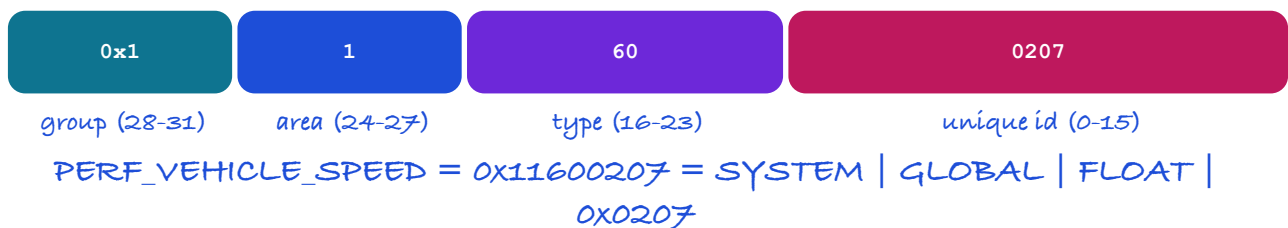
BULLET POINTS

- Apps → Car API → CarService → VHAL → MCU → buses.
- Android Auto projects; AAOS is the OS.
- GAS = Google apps licensed by the OEM.
- Safety-relevant things (camera, chimes) start early, outside Java.
- Headless system user 0; drivers are secondary users.

3 The Vehicle HAL and vehicle properties

everything is a property

VHAL exposes the car as a list of **properties**. Each has a 32-bit ID, a value type, an area, an access mode and a change mode. Google defines hundreds of standard ones (VehiclePropertyIds); OEMs add **vendor** properties.



Field	Values you must know
Group	SYSTEM 0x10000000 (Google standard), VENDOR 0x20000000 (OEM-specific)
Area type	GLOBAL 0x01, WINDOW 0x03, MIRROR 0x04, SEAT 0x05, DOOR 0x06, WHEEL 0x07 (shifted << 24)
Value type	STRING 0x10, BOOLEAN 0x20, INT32 0x40, INT32_VEC 0x41, INT64 0x50, FLOAT 0x60, FLOAT_VEC 0x61, BYTES 0x70, MIXED 0xE0 (shifted << 16)
Access	READ, WRITE, READ_WRITE
Change mode	STATIC (read once, e.g. INFO_MAKE), ON_CHANGE (gear, doors), CONTINUOUS (speed, rpm: subscribe with a sample rate between minSampleRate and maxSampleRate)

Using CarPropertyManager

```
val car = Car.createCar(context, null, CAR_WAIT_TIMEOUT_WAIT_FOREVER) { car, ready ->
    if (!ready) return@createCar // CarService died
    val pm = car.getCarManager(Car.PROPERTY_SERVICE) as CarPropertyManager
    // read once
    val make = pm.getProperty<String>(VehiclePropertyIds.INFO_MAKE, 0).value
    // subscribe (API 35+: subscribePropertyEvents)
    pm.registerCallback(callback, VehiclePropertyIds.PERF_VEHICLE_SPEED,
        CarPropertyManager.SENSOR_RATE_UI)
    // write (needs the property's write permission)
    pm.setProperty(Float::class.java, VehiclePropertyIds.HVAC_TEMPERATURE_SET,
        areaId, 21.5f)
}
```

REAL LIFE

When you change the temperature knob in a Polestar 2, the HVAC app calls `setProperty` on `HVAC_TEMPERATURE_SET`. The Tier-1's VHAL implementation turns it into a signal in a CAN frame to the climate ECU, which moves the blend-door actuator. When the ECU confirms, VHAL emits an `ON_CHANGE` event and every subscribed UI updates.

SEE IT MOVE: IN THE APP

Lesson 3 **Vehicle HAL**: each row flashes when a change event arrives (CONTINUOUS rows flash constantly while you drive in the Cluster lesson). Tap a row to split its hex ID into coloured fields.

SHARPEN YOUR PENCIL

Decode these IDs (answers at the end of the book):

(a) `0x15600503` (b) `0x11400400` (c) `0x16200B02` (d) `0x21400001`

THERE ARE NO DUMB QUESTIONS

Q: What happens if I subscribe to a CONTINUOUS property at 100 Hz but `maxSampleRate` is 10?

A: The rate is clamped to what the config allows. Always read `CarPropertyConfig` first.

Q: How do I test without a car?

A: AAOS emulator Extended Controls (Car data), the VHAL emulator/reference VHAL, or `adb shell cmd car_service inject-vhal-event <propId> <value>`. In unit tests, fake the data source (like this app's `SimulatedVehicleDataSource`).

Q: AIDL or HIDL VHAL?

A: Android 13 introduced the AIDL VHAL (`android.hardware.automotive.vehicle.IVehicle`); HIDL (2.0) is legacy. New projects implement AIDL.

WATCH IT!

Units! Speed is m/s, temperatures °C, fuel in millilitres, EV battery in Wh, distance in mm. Converting at the UI layer (not in VHAL) keeps the platform SI-consistent.

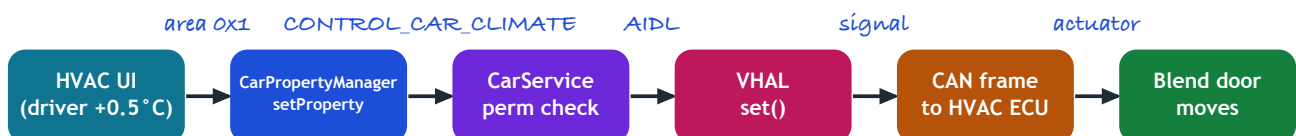
BULLET POINTS

- Property = ID + type + area + access + change mode.
- ID bits: group|area|type|id.
- STATIC read once; ON_CHANGE events; CONTINUOUS with rate.
- Vendor properties use group 0x2.
- Re-subscribe after CarService restart.

4 HVAC and area properties

same property, different seat

Area properties exist once per **area ID**. For HVAC the area type is SEAT, so area IDs are bitmasks of VehicleAreaSeat: ROW_1_LEFT = 0x0001, ROW_1_CENTER = 0x0002, ROW_1_RIGHT = 0x0004, ROW_2_LEFT = 0x0010, ROW_2_CENTER = 0x0020, ROW_2_RIGHT = 0x0040...



WATCH IT!

The OEM decides how seats are grouped. On a 2-zone car the driver area might be 0x31 (ROW_1_LEFT | ROW_2_LEFT | ROW_2_CENTER). Never hard-code 0x1: read `getCarPropertyConfig(id).areaIds` and pick the one containing your seat bit. This app does exactly that in `CarApiVehicleDataSource.setHvac()`.

HVAC property	Type / area	Real-life use
HVAC_TEMPERATURE_SET	FLOAT / SEAT	Dual-zone temperature
HVAC_FAN_SPEED	INT32 / SEAT	Blower level
HVAC_AC_ON	BOOLEAN / SEAT	Compressor on/off
HVAC_DEFROSTER	BOOLEAN / WINDOW	Front/rear defrost
HVAC_SEAT_TEMPERATURE	INT32 / SEAT	Heated (+) / ventilated (-) seats
HVAC_POWER_ON	BOOLEAN / SEAT	Master switch; other HVAC props may become unavailable when off

BRAIN POWER

Reading speed only needs CAR_SPEED (dangerous), but writing temperature needs CONTROL_CAR_CLIMATE (signature|privileged). Why would Google make *writing* the climate stricter than *reading* speed?

REAL LIFE

Remote pre-conditioning (cool the car from your phone before you get in) travels phone → OEM cloud → telematics unit (TCU) → vehicle bus → HVAC ECU, often while the head unit is asleep. AAOS is not even involved; the TCU and the HVAC ECU are.

SEE IT MOVE: IN THE APP

Lesson 4 **HVAC**: the fan's rotation speed and airflow particles follow HVAC_FAN_SPEED; the zone card fades from blue to red with temperature. On a real AAOS build you'll see the SecurityException message because the app is not privileged.

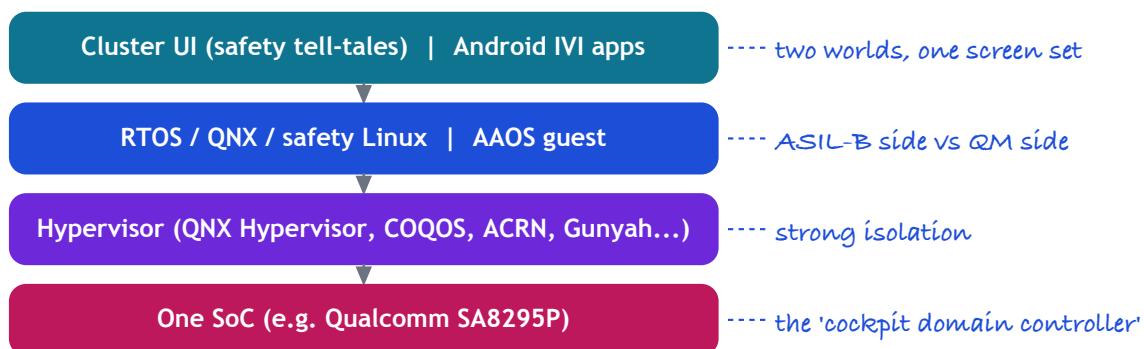
BULLET POINTS

- Area ID = seat bitmask for HVAC.
- Read areaIds from config; never hard-code.
- Writing HVAC is signature|privileged.
- HVAC_POWER_ON gates other HVAC properties.

5 The instrument cluster and functional safety

the screen behind the steering wheel

The **cluster** shows speed, rpm, gear, range and **tell-tales** (warning lamps: brake, ABS, airbag, seat belt, engine). Some tell-tales are legally required (UN R121) and safety-relevant.



A typical digital cockpit: safety and Android share one chip, isolated by a hypervisor

ISO 26262 in one table

Level	Meaning	Example
QM	Quality managed, no safety requirement	Media player, navigation UI
ASIL A	Lowest safety integrity	Rear lights
ASIL B	Low-medium	Cluster tell-tales, rear camera display
ASIL C	Medium-high	Adaptive cruise control
ASIL D	Highest	Braking, steering, airbag deployment

REAL LIFE

Mercedes MBUX Hyperscreen and many Harman/Visteon digital cockpits run cluster + IVI + passenger display on a single chip with a hypervisor. If Android crashes, the speedometer keeps working.

SEE IT MOVE: IN THE APP

Lesson **2 Instrument Cluster**: needles use a low-stiffness spring so they sweep smoothly between 10 Hz samples; the gear letter slides via `AnimatedContent`; indicators blink with an infinite transition. Try shifting to P at 60 km/h: the simulated transmission interlock refuses.

THERE ARE NO DUMB QUESTIONS

Q: Can I write the cluster in Android?

A: AAOS supports a cluster display (`ClusterHomeService`, cluster activity), often used for navigation maps in the cluster. The legally required tell-tales usually stay on the safety side.

Q: ASIL vs SIL?

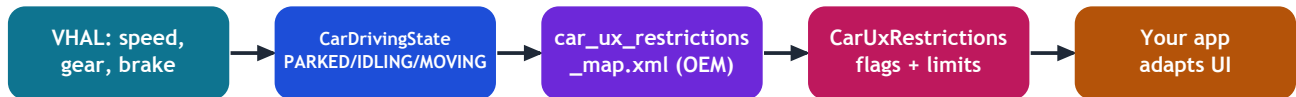
A: SIL is from IEC 61508 (general industry). ASIL is the automotive adaptation in ISO 26262.

BULLET POINTS

- Tell-tales are safety-relevant; IVI is QM.
- Hypervisors isolate safety from Android.
- ASIL A-D + QM (ISO 26262).
- Animate needles with springs for realism.

6 Driver distraction and UX restrictions

eyes on the road, not on your app



Restriction flag	What your UI must do
NO_VIDEO	Stop/hide video; show a 'not available while driving' message
NO_KEYBOARD	Disable text input; offer voice
LIMIT_STRING_LENGTH	Truncate text to getMaxRestrictedStringLength() (often 120)
LIMIT_CONTENT	Cap items (getMaxCumulativeContentItems) and depth (getMaxContentDepth)
NO_SETUP	No account sign-in / settings flows
NO_TEXT_MESSAGE	Do not show message bodies; read them aloud instead
NO_DIALPAD / NO_FILTERING / NO_VOICE_TRANSCRIPTION	Hide dial pad / list filters / live transcription

```
val m = car.getCarManager(Car.CAR_UX_RESTRICTION_SERVICE)
    as CarUxRestrictionsManager
m.registerListener { r ->
    val noVideo = CarUxRestrictions.UX_RESTRICTIONS_NO_VIDEO
    videoAllowed = r.activeRestrictions and noVideo == 0
    maxItems      = r.maxCumulativeContentItems
}
```

REAL LIFE

US NHTSA visual-manual guidelines: single glances ≤ 2 s, total task ≤ 12 s. That is where limits like '6 list items' and 'no keyboard while moving' come from. Europe uses similar ESoP guidelines.

WATCH IT!

Apps must react to **restrictions**, not to speed. The OEM may restrict even at 0 km/h (IDLING in gear) or allow more on a passenger screen.

An activity without `distractionOptimized=true` is covered by the `ActivityBlockingActivity` as soon as the car moves.

SEE IT MOVE: IN THE APP

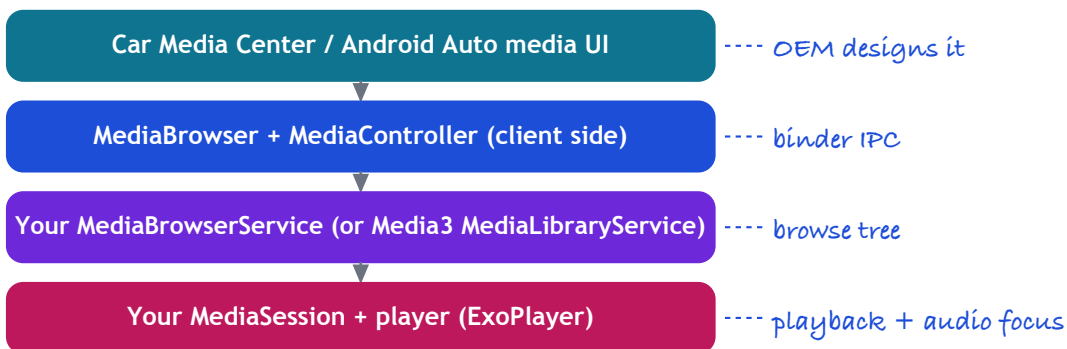
Lesson 5 **Driver Distraction**: drag speed above zero; restriction chips bounce in, the video gets a lock overlay, the keyboard disables, the list collapses and the message truncates. On AAOS with Real Car API selected it listens to the real `CarUxRestrictionsManager`.

BULLET POINTS

- Driving state → OEM map → restrictions.
- Obey flags + limits, not raw speed.
- Non-DO activities get blocked while moving.
- Templates are DO by design.

7 Media apps in the car

you bring the music, the car brings the UI



Callback / API	Job
onGetRoot(package, uid, hints)	Validate the caller; return root id + extras (content style hints)
onLoadChildren(parentId)	Return browsable (folders) and playable (tracks) items
MediaSession.Callback	onPlay, onPause, onPlayFromMediaId, onSkipToNext, onPlayFromSearch (voice!)
PlaybackState actions	Declare which buttons the car shows; set position + speed for the seek bar
Content style hints	CONTENT_STYLE_BROWSABLE_HINT / PLAYABLE_HINT: grid vs list

REAL LIFE

Spotify, YouTube Music, Audible, Pocket Casts all look 'native' in Volvo, GM, Renault because the OEM's media template draws them. Steering-wheel next/prev buttons are routed as key events to the active MediaSession.

SEE IT MOVE: IN THE APP

Lesson **6 Media** connects to this app's own CarMediaBrowserService with MediaBrowserCompat (exactly like the car). Tap a track: the equalizer animates, the title slides in and the seek bar is extrapolated from PlaybackState (position + update time × speed).

THERE ARE NO DUMB QUESTIONS

Q: Media3 or MediaBrowserServiceCompat?

A: New apps should use Media3 (MediaLibraryService + MediaSession + ExoPlayer). The concepts are the same; this app uses the compat API because it maps 1:1 to what the car calls.

Q: How does voice work ('Play jazz on Spotify')?

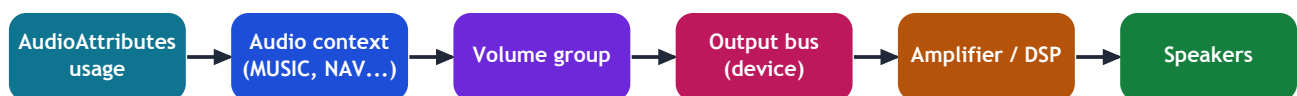
A: The assistant calls onPlayFromSearch(query, extras). Support it: cars rely on voice while driving.

BULLET POINTS

- MediaBrowserService = tree; MediaSession = control.
- Validate callers in onGetRoot.
- Actions decide which buttons appear.
- Support search for voice.

8 Car audio: contexts, zones and focus

who gets the speakers?



CarAudioService arbitrates focus using an **interaction matrix**:

Holder ↓ / Requester →	Music	Navigation	Call	Voice assistant
Music	exclusive	concurrent (duck)	exclusive	exclusive
Navigation	concurrent	exclusive	exclusive	exclusive
Call	reject	concurrent (duck)	exclusive	reject
Voice assistant	reject	concurrent	exclusive	exclusive

(Simplified from the AOSP defaults; OEMs customise it.)

REAL LIFE

Maps says 'turn left' → music ducks. A call comes in → music pauses. Seat-belt chime → always plays, often on a separate safety path. BMW 7 Series rear seats can listen to a different stream on headphones: a second **audio zone**.

WATCH IT!

In many cars ducking is done by the **external amplifier/DSP**, not by Android's mixer. That is why AAOS uses fixed-volume output buses and lets the hardware handle gain.

SEE IT MOVE: IN THE APP

Lesson 7 **Car Audio**: mixer bars animate to show exclusive (music → o), concurrent (music ducks to 30 %) and reject. It also makes a real `AudioFocusRequest` so you can inspect `adb shell dumpsys car_service / dumpsys audio`.

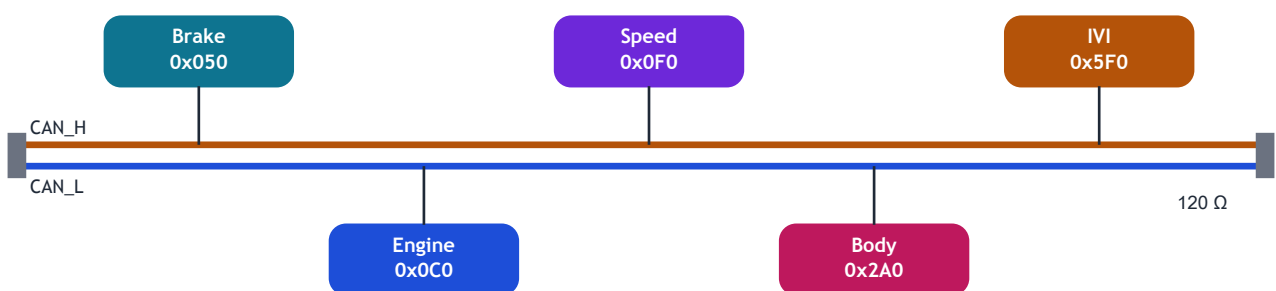
BULLET POINTS

- Usage → context → volume group → bus.
- Exclusive / concurrent / reject.
- Zones = independent audio per occupant.
- Always request focus with the right usage.

9 Vehicle networks and diagnostics

CAN, LIN, FlexRay, Ethernet, UDS

Bus	Speed	Payload	Used for
LIN	≤ 20 kbps	8 bytes	Cheap sensors/actuators: window switches, mirrors, seat motors
CAN (HS)	125 kbps - 1 Mbps (500 k typical)	8 bytes	Powertrain, body, chassis messages
CAN FD	up to ~5-8 Mbps data phase	64 bytes	More data + room for SecOC MACs
CAN XL	up to ~20 Mbps	2048 bytes	Bridge between CAN and Ethernet worlds
FlexRay	10 Mbps, time-triggered	254 bytes	Deterministic chassis (older premium cars)
Automotive Ethernet	100BASE-T1 / 1000BASE-T1 / multi-gig	large	Cameras, ADAS, backbone, OTA, DoIP



Lower ID = higher priority. The brake always wins.

Arbitration, bit by bit

All nodes start sending their ID at the same time, MSB first. The bus is a **wired-AND**: a dominant 0 overrides a recessive 1. A node that sends 1 but reads 0 knows it lost and becomes a receiver. The winner never notices the collision, so no bandwidth is wasted.

000 0101 0000

brake 0x050

000 1111 0000

speed 0x0F0

They differ at bit 3 (from MSB): brake sends 0, speed sends 1 → speed backs off

Signals and DBC files

A frame's 8 bytes carry several **signals**. The DBC file (communication matrix) says: signal VehicleSpeed is in frame 0x0F0, start bit 0, length 16, little-endian, factor 0.01, offset 0, unit km/h. Physical value = raw × factor + offset.

```
raw    = 0x3039          // bytes 39 30 on the wire (little-endian)
speed  = 12345 * 0.01     // = 123.45 km/h      (see CanBus.encodeSpeed / decodeSpeed)
```

Diagnostics

Standard	What	Example
OBD-II (SAE J1979 / ISO 15031)	Legally required emissions diagnostics via the OBD port	Mode 01 PID 0x0D = vehicle speed; DTC P0300 misfire
UDS (ISO 14229)	Unified Diagnostic Services for all ECUs	0x10 session control, 0x22 read DID, 0x27 security access, 0x29 authentication, 0x2E write DID, 0x31 routine, 0x34/0x36/0x37 flash download, 0x19 read DTCs
ISO-TP (ISO 15765-2)	Transport layer to send > 8 bytes over CAN	First frame + consecutive frames
DoIP (ISO 13400)	Diagnostics over IP (Ethernet)	Fast ECU flashing at the factory/OTA

REAL LIFE

A cheap Bluetooth OBD-II dongle + the Torque app reads live engine data from almost any car made after 2001 (EU) / 1996 (US). Workshops flash ECUs using UDS 0x34/0x36/0x37 after unlocking with 0x27.

SEE IT MOVE: IN THE APP

Lesson **8 CAN Bus**: a frame travels from the Speed ECU in both directions along CAN_H/CAN_L; the candump shows the live encoded speed. In the arbitration demo pick ECUs and watch the IDs revealed bit by bit; losers turn grey at the bit where they lost.

SHARPEN YOUR PENCIL

- (e) Frames 0x2A0 and 0x2A4 start together. Which wins, and at which bit?
- (f) Raw speed bytes on the wire are E8 03. What is the speed with factor 0.01?

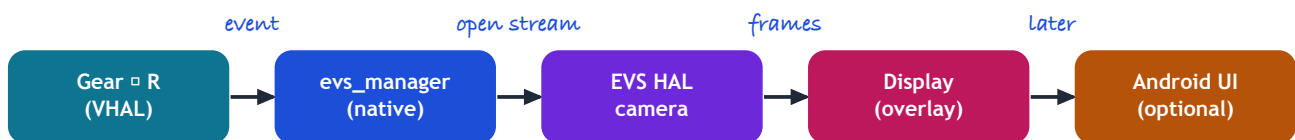
BULLET POINTS

- LIN < CAN < CAN FD < Ethernet.
- Lowest ID wins non-destructively.
- DBC maps bits to physical signals.
- UDS services: 0x10, 0x22, 0x27, 0x2E, 0x19, 0x34-37.
- CAN has no authentication → SecOC (book 03).

10 Cameras, park assist, EVS and ADAS

seeing around the car

Ultrasonic sensors in the bumpers send a 40-50 kHz ping and time the echo: distance = speed of sound (≈ 343 m/s) $\times$ time / 2. **Cameras** give rear/surround views. **Radar and lidar** feed ADAS features like adaptive cruise and emergency braking (on dedicated ADAS ECUs, not Android).



REAL LIFE

US FMVSS 111 requires a rear-view image within **2 seconds** of shifting into reverse on all new light vehicles since May 2018. That is why AAOS has the Extended View System (EVS): a native stack started early by init that can show the camera before the Java framework has finished booting.

SAE level	Who drives	Example
0	Driver; warnings only	Parking beeps, lane departure warning
1	Driver + one assist	Adaptive cruise OR lane keep
2	Driver supervises steering + speed assist	Tesla Autopilot, GM Super Cruise, Ford BlueCruise
3	System drives in limited conditions; driver takes over on request	Mercedes Drive Pilot
4	No driver needed in a defined area	Waymo robotaxis
5	Everywhere	Not available yet

SEE IT MOVE: IN THE APP

Lesson 9 Park Assist: ultrasonic arcs expand from the rear bumper toward the wall and change green → amber → red; enable beep sound in R and the beep interval shrinks as the obstacle gets closer.

BULLET POINTS

- distance = $343 \text{ m/s} \times t / 2$.
- EVS = early native camera path.
- FMVSS 111: image within 2 s.
- ADAS runs on safety ECUs; AAOS only visualises.

11 Electric vehicles and charging

range anxiety is a UX problem

Property	Meaning
EV_BATTERY_LEVEL (Wh)	Energy left (divide by INFO_EV_BATTERY_CAPACITY for %)
EV_CHARGE_PORT_OPEN / CONNECTED	Flap open? Cable plugged in?
EV_BATTERY_INSTANTANEOUS_CHARGE_RATE (mW)	Positive = charging, negative = discharging
RANGE_REMAINING (m)	OEM-computed range
EV_CHARGE_PERCENT_LIMIT / EV_CHARGE_SWITCH	Charge target and start/stop charging

Why fast charging slows down after 80 %

Lithium-ion cells are charged in two phases: **constant current (CC)** until the cell voltage reaches its maximum, then **constant voltage (CV)** where the current tapers to avoid lithium plating and heat. So 10 → 80 % may take ~20-30 minutes and 80 → 100 % almost as long.

REAL LIFE

Google Maps on AAOS (Polestar, Volvo) plans charging stops using live EV_BATTERY_LEVEL and predicts arrival SoC; it can pre-condition the battery so it charges faster on arrival. Charging-network apps (ChargePoint, PlugShare) use the Car App Library CHARGING category.

SEE IT MOVE: IN THE APP

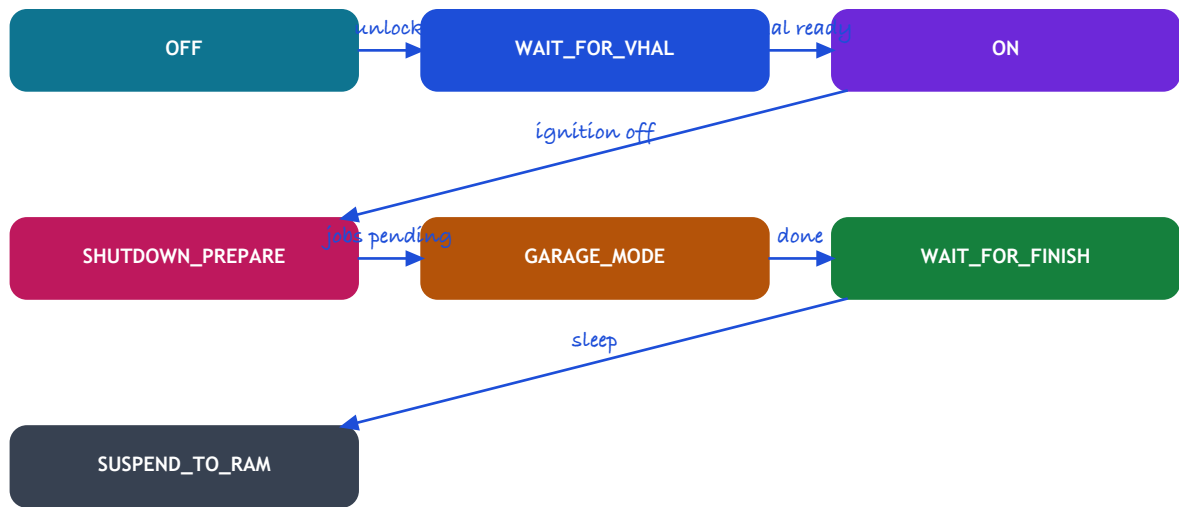
Lesson **10 EV & Charging**: plug in; the battery glows and fills, the dot slides along the CC/CV curve and the label switches from 'Fast charging (CC)' to 'Tapering (CV)' at 80 %. Try plugging in while driving: rejected.

BULLET POINTS

- Battery level in Wh; % = level / capacity.
- CC then CV; 80 % knee.
- Range = usable energy / consumption, adjusted for HVAC & temperature.
- Charging apps = Car App Library CHARGING category.

12 Power management

a car is 'off' for 95 % of its life



Simplified CPMS states (see PowerStateMachine.kt)

The **Vehicle MCU** is always powered and decides when the Android SoC sleeps or wakes. It talks to CarPowerManagementService through the VHAL properties AP_POWER_STATE_REQ and AP_POWER_STATE_REPORT.

Concept	What it means
Suspend-to-RAM	Keep DRAM powered, everything else off; resume in ~1-2 s instead of a cold boot
Suspend-to-disk (hibernation)	Write RAM to storage; zero drain, slower resume
Garage Mode	After the driver leaves: run idle JobScheduler jobs (updates, sync) for a limited time
Power policies	CarPowerPolicy turns components (audio, display, Wi-Fi, BT) on/off per state
Silent mode	Boot with screens/audio off, e.g. for an OTA at night

BRAIN POWER

The 12 V battery has maybe 60 Ah. If the head unit drew 2 A all night it would be half empty by morning. What would the driver experience?

SEE IT MOVE: IN THE APP

Lesson **11 Power**: only valid events are shown for the current state; the active state pulses. Go ON → Ignition off → Idle jobs pending → Jobs finished → Enter deep sleep → Unlock.

BULLET POINTS

- The MCU is the boss of power.
- Suspend-to-RAM = fast wake.
- Garage Mode runs idle jobs after the drive.
- Listen to CarPowerManager to save state.

13 Over-the-air (OTA) updates

update a car without a workshop



Term	Meaning
A/B (seamless) update	Two copies of partitions; install to the inactive one while driving
update_engine	Android daemon that applies A/B payloads (full or delta)
AVB + dm-verity	Verified boot checks the new slot's signature and every block at runtime
Virtual A/B	Snapshots save storage space instead of full duplicate partitions
Uptane	Secure update framework for ECUs (director + image repositories, compromise-resilient)
UNECE R156 / SUMS	EU-mandated Software Update Management System for OEMs

REAL LIFE

Tesla, Polestar, Ford and GM ship new features and fixes by OTA, sometimes including changes to driving features, not only infotainment. A recall that once meant a workshop visit can now be an overnight software update.

SEE IT MOVE: IN THE APP

Lesson **12 OTA**: start an update; the inactive slot glows amber while written, then the slots swap. Turn on 'Simulate a broken image' to watch the rollback. Drive during **READY_TO_REBOOT**: the reboot waits until you park.

WATCH IT!

Never reboot a head unit while the car is moving: the driver loses the rear camera, navigation and warnings. Always gate on driving state and ask the user.

BULLET POINTS

- Install to inactive slot; reboot later.
- Verified boot + rollback = no bricking.
- Download in Garage Mode.
- R156 makes SUMS mandatory in the EU.

14 Multiple displays, users and occupant zones

one car, many screens, many people

CarOccupantZoneManager maps each occupant (driver, front passenger, rear seats) to displays, an audio zone and an Android user. Since Android 14, AAOS supports **multi-user multi-display (MUMD)**: several users logged in concurrently on different screens.

Zone	Display	User	Video while moving?
Driver	Cluster + centre	Current driver user (e.g. 10)	No (UX restrictions)
Front passenger	Passenger display	Visitor or passenger user	Yes, if not visible to driver
Rear left / right	Rear-seat screens	Separate users	Yes

REAL LIFE

Mercedes EQS/EQE passenger screens play video while driving; a driver-facing camera dims the passenger screen if the driver looks at it. Driver profiles switch automatically based on the key fob or phone key.

SEE IT MOVE: IN THE APP

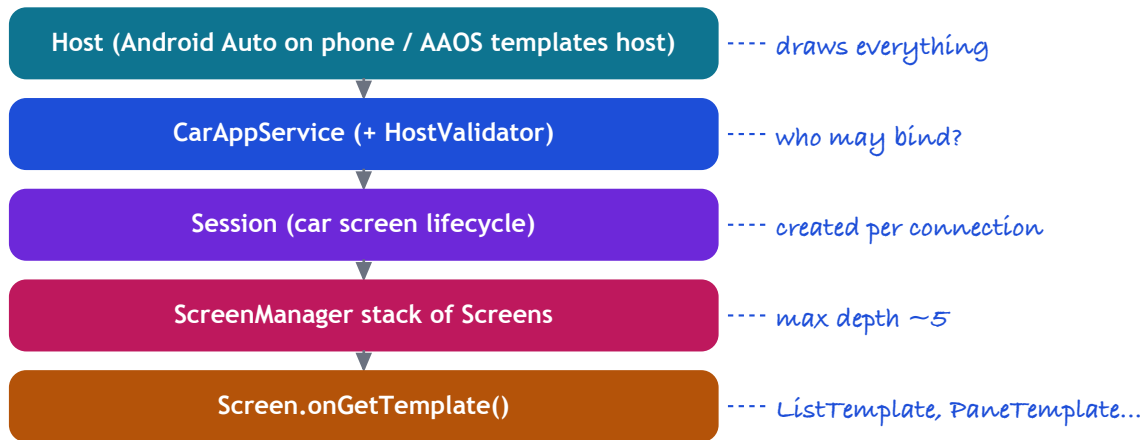
Lesson **14 Multi-Display**: the focus ring springs from seat to seat, showing each zone's display, user, audio zone and video policy.

BULLET POINTS

- Occupant zone → displays + audio zone + user.
- User 0 is headless system user.
- MUMD: concurrent users (Android 14+).
- Restrictions can differ per display.

15 The Car App Library

templates: safe by construction



Category	Typical templates	Example apps
Navigation	NavigationTemplate, MapWithContentTemplate	Waze, Sygic
POI: charging / parking	PlaceListMapTemplate, ListTemplate, PaneTemplate	ChargePoint, SpotHero
IoT	GridTemplate, ListTemplate	Google Home (garage door)
Weather, messaging, media (newer API levels)	ListTemplate, MessageTemplate...	Weather apps, chat

WATCH IT!

Templates have limits (rows per list, steps per task, refresh quota). Design for them.

Use a strict HostValidator in release builds; ALLOW_ALL is for debugging only.

On AAOS, template apps also need the app-automotive artifact and CarAppActivity.

SEE IT MOVE: IN THE APP

Lesson **15 Car App Library** simulates a host: push screens with row taps (slide animation) until the depth limit blocks you. Then run the real ChargingStationCarAppService on the Desktop Head Unit.

BULLET POINTS

- CarAppService → Session → Screen → Template.
- Host renders; app supplies models.
- Categories declared in the manifest.
- Test on DHU / AAOS emulator.

16 Connectivity: Bluetooth, projection, telematics, V2X

Tech	What	Interview detail
Bluetooth HFP	Hands-free calls	Car is HF unit, phone is AG; SCO audio
A2DP + AVRCP	Stream music + remote control & metadata	Car is A2DP sink
PBAP / MAP	Phonebook and messages	Needs user consent on phone
Android Auto / CarPlay	Phone projection (USB or wireless over Wi-Fi)	Competes with native AAOS apps
TCU (telematics)	Cellular modem ECU: eCall, remote lock, OTA, fleet data	eCall mandatory in EU since 2018
V2X (C-V2X / DSRC)	Vehicle-to-vehicle/infrastructure messages	Signed with PKI certificates; low latency
Digital Key (CCC 3.0)	Phone as key via NFC, BLE, UWB	UWB ranging stops relay attacks

BRAIN POWER

Why does the car need UWB for a phone key when BLE already works? (Hint: an attacker with two radios standing near you and near your car.)

17 AUTOSAR, SDV and how cars are tested

	AUTOSAR Classic	AUTOSAR Adaptive
Runs on	Microcontrollers (RTOS, static)	Microprocessors (POSIX, e.g. Linux/QNX)
Communication	Signals over CAN/LIN/FlexRay (COM stack)	Service-oriented over Ethernet (SOME/IP, DDS)
Config	Static at build time	Dynamic, updatable
Used for	Brakes, engine, body	Central compute, ADAS, gateways

Test level	What	Tools
Unit	Pure logic on JVM	JUnit, MockK, Robolectric
SIL	Software-in-the-loop: whole stack with simulated vehicle	AAOS emulator + fake VHAL
HIL	Hardware-in-the-loop: real head unit + simulated car network	Vector CANoe, dSPACE
Compliance	Android compatibility	CTS, VTS, CTS-Verifier, GAS test suites
Vehicle	Real drives, EMC, climate chambers	Test fleets

INTERVIEW TIP

If asked 'how would you test this feature?', walk the pyramid: unit → SIL on emulator with injected VHAL events → HIL with CANoe replaying real traces → in-vehicle validation.

Sharpen your pencil: answers

- (a) `0x15600503` = SYSTEM | SEAT | FLOAT | `0x0503` → HVAC_TEMPERATURE_SET.
- (b) `0x11400400` = SYSTEM | GLOBAL | INT32 | `0x0400` → GEAR_SELECTION.
- (c) `0x16200B02` = SYSTEM | DOOR | BOOLEAN | `0x0B02` → DOOR_LOCK.
- (d) `0x21400001` = VENDOR | GLOBAL | INT32 | `0x0001` → an OEM-specific property.
- (e) `0x2A0` = `010 1010 0000` and `0x2A4` = `010 1010 0100`. They differ at the bit with value 4 (the 9th bit sent). `0x2A0` sends dominant 0 there and wins.
- (f) Little-endian `E8 03` → `0x03E8` = 1000 → $1000 \times 0.01 = 10.00$ km/h.

Glossary

Term	Meaning
AAOS	Android Automotive OS
ADAS	Advanced Driver Assistance Systems
ASIL	Automotive Safety Integrity Level (ISO 26262)
BCM	Body Control Module
CAN	Controller Area Network
CPMS	Car Power Management Service
DBC	CAN database file describing frames/signals
DHU	Desktop Head Unit (Android Auto emulator)
DO	Distraction Optimized
DoIP	Diagnostics over IP
ECU	Electronic Control Unit
EVS	Extended View System (cameras)
GAS	Google Automotive Services
HIL / SIL	Hardware / Software in the loop testing
IVI	In-Vehicle Infotainment
MCU	Microcontroller (vehicle MCU is always on)
OEM	Car maker
SDV	Software-Defined Vehicle
SecOC	Secure Onboard Communication
TCU	Telematics Control Unit
Tier-1	Direct supplier to OEM (Harman, Bosch...)
UDS	Unified Diagnostic Services (ISO 14229)
UXR	UX Restrictions
VHAL	Vehicle Hardware Abstraction Layer